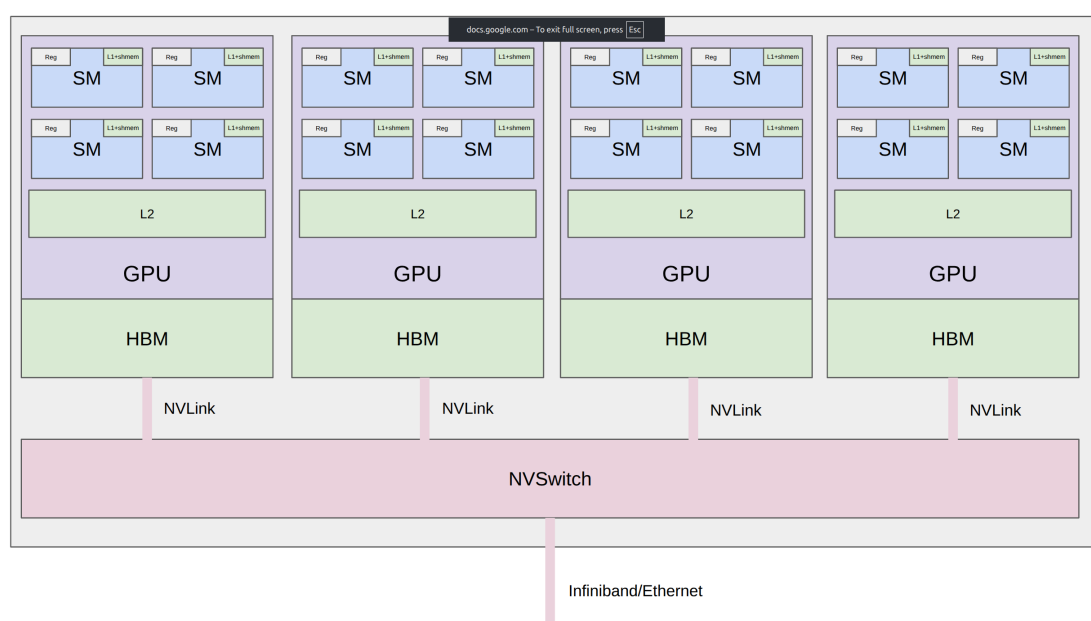


## 课程笔记

# CS336 2026 Lecture 7: Parallelism 与分布式训练基础

基于 Stanford CS336 Spring 2026 官方可执行讲义重新整理

2026 年春季



课程/频道: Stanford CS336

发布日期: Spring 2026

材料类型: 官方可执行讲义 lecture07-slides.py

课程主页: <https://cs336.stanford.edu/>

项目主页: <https://github.com/hqh1025/ai-course-notes>

# 目录

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>1 本讲主线：从单卡局部性到集群局部性</b>                         | <b>3</b>  |
| 1.1 为什么要多 GPU                                      | 4         |
| 1.2 泛化的层次结构                                        | 5         |
| 1.3 本章小结                                           | 5         |
| <b>2 Collective Operations：分布式训练的通信积木</b>          | <b>5</b>  |
| 2.1 rank、world size 与 process group                | 5         |
| 2.2 术语消化：从基础原语到工作马                                 | 6         |
| 2.3 broadcast、scatter、gather、reduce                | 6         |
| 2.4 all-gather、reduce-scatter、all-reduce           | 8         |
| 2.5 all-to-all：MoE 的通信核心                           | 10        |
| 2.6 本章小结                                           | 10        |
| <b>3 硬件互连：拓扑决定可承受的通信频率</b>                         | <b>10</b> |
| 3.1 从家用拓扑到数据中心拓扑                                   | 10        |
| 3.2 带宽数量级和路径差异                                     | 12        |
| 3.3 RDMA、InfiniBand 与 RoCE                         | 12        |
| 3.4 NVIDIA Collective Communication Library (NCCL) | 12        |
| 3.5 本章小结                                           | 13        |
| <b>4 torch.distributed：把 collective 写成代码</b>       | <b>13</b> |
| 4.1 初始化和 process group                             | 13        |
| 4.2 all-reduce 示例                                  | 13        |
| 4.3 reduce-scatter + all-gather 示例                 | 14        |
| 4.4 barrier、同步和 deadlock                           | 14        |
| 4.5 本章小结                                           | 14        |
| <b>5 通信 Benchmark：测的是带宽，也是算法</b>                   | <b>15</b> |
| 5.1 为什么要单独测 collective                             | 15        |
| 5.2 带宽估算公式                                         | 15        |
| 5.3 常见测量陷阱                                         | 15        |
| 5.4 本章小结                                           | 16        |
| <b>6 Data Parallelism：沿 batch 维切分</b>              | <b>16</b> |
| 6.1 基本策略                                           | 16        |
| 6.2 通信账本                                           | 17        |
| 6.3 为什么引出 FSDP/Zero                                | 17        |
| 6.4 本章小结                                           | 18        |
| <b>7 Tensor Parallelism：沿 width 维切分</b>            | <b>18</b> |
| 7.1 基本策略                                           | 18        |
| 7.2 列切、行切和通信位置                                     | 20        |
| 7.3 反向传播为什么更复杂                                     | 20        |
| 7.4 本章小结                                           | 20        |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>8 Pipeline Parallelism: 沿 depth 维切分</b>      | <b>20</b> |
| 8.1 基本策略 . . . . .                              | 20        |
| 8.2 bubble 的直觉公式 . . . . .                      | 22        |
| 8.3 为什么 pipeline 能跨较慢链路 . . . . .               | 22        |
| 8.4 本章小结 . . . . .                              | 22        |
| <b>9 三种并行策略的比较与组合</b>                           | <b>22</b> |
| 9.1 切分对象决定通信形态 . . . . .                        | 22        |
| 9.2 recompute、memory、communication 三角 . . . . . | 23        |
| 9.3 JAX/TPU 路线为什么看起来不同 . . . . .                | 23        |
| 9.4 本章小结 . . . . .                              | 23        |
| <b>10 工程 Checklist: 写分布式训练代码前先问什么</b>           | <b>24</b> |
| 10.1 状态在哪里 . . . . .                            | 24        |
| 10.2 通信何时发生 . . . . .                           | 24        |
| 10.3 硬件是否匹配策略 . . . . .                         | 24        |
| 10.4 本章小结 . . . . .                             | 24        |
| <b>11 拓展阅读</b>                                  | <b>25</b> |
| <b>12 Summary: 总结与延伸</b>                        | <b>25</b> |

# 1 本讲主线：从单卡局部性到集群局部性

Lecture 5 和 Lecture 6 讲的是单张 GPU 内部的并行：如何让 tensor core 吃饱，如何用 fusion 和 tiling 减少 HBM 往返，如何用 Triton 把一个算子写成更接近硬件的数据搬运程序。Lecture 7 把同一个问题放大到多 GPU 和多节点。官方源文件把这一部分命名为 **Part 1: building blocks of distributed communication/computation**，然后再进入 **Part 2: distributed training**。这两个名字很重要：先学通信语言，再学训练策略。

## 术语消化：本讲第一批系统词

| 术语         | 字面含义                                                                                                        | 本讲中的作用                                                           |
|------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| HBM        | High Bandwidth Memory，高带宽显存，是 GPU 上的 DRAM 形态。                                                               | 单卡上参数、激活和中间张量主要存放在这里；上一讲优化 HBM traffic，本讲则把瓶颈扩展到 GPU 间通信。        |
| sharding   | 分片；把原本完整复制的数据切开，让每张 GPU 只存其中一部分。                                                                            | ZeRO/FSDP、tensor parallel、pipeline parallel 都是在不同对象上做 sharding。  |
| ZeRO       | Zero Redundancy Optimizer；在 data parallel rank 之间分片 optimizer state、gradients、parameters，常按 stage 1/2/3 区分。 | 本讲先用 all-gather 和 reduce-scatter 建立直觉，下一讲会系统解释 ZeRO/FSDP 如何降低显存。 |
| collective | 集合通信；所有 rank 共同进入同一个通信原语。                                                                                   | 分布式训练的基本词汇，后面所有并行策略都能拆成若干 collectives。                           |

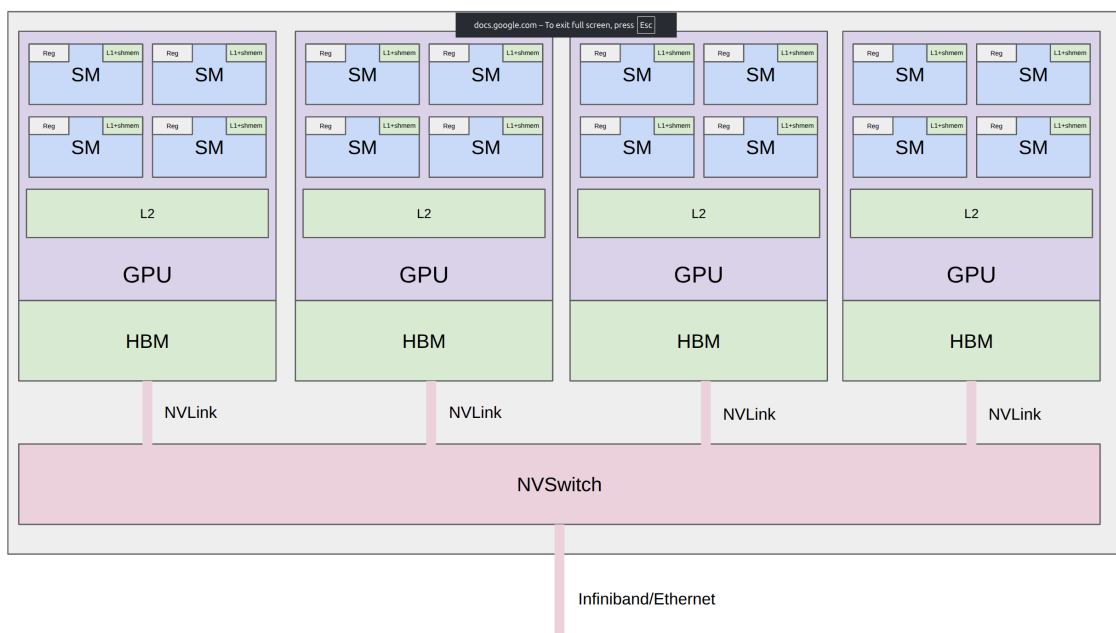


图 1: 现代 GPU 节点和集群拓扑: 节点内通过 NVLink/NVSwitch, 节点间通过 HCA/NIC 和 InfiniBand/RoCE。

来源: CS336 Spring 2026 Lecture 7 官方可执行讲义图像。

### 读图: 这张节点拓扑图应该怎么看

先看最内层: 每张 GPU 旁边都有自己的 HBM, 计算单元访问本卡 HBM 比访问其他 GPU 的数据快得多。再看节点内互连: NVLink/NVSwitch 把多张 GPU 连成一个高带宽域, 适合频繁交换激活或分片参数。最后看节点外: HCA/NIC、交换机、跨机链路把节点连成集群, 带宽和延迟都明显更差。因此同样是“通信”, 在一台机器里和跨机器的含义不同; tensor parallel 这种每层通信的策略必须尽量放在快互连上, data/pipeline parallel 这种通信频率较低的策略更能承受慢链路。

### 统一主题: compute 离 data 总是有距离

单卡性能工程是在 HBM、cache、register 之间安排数据; 多卡性能工程是在 GPU、node、pod 之间安排数据。上一讲用 fusion/tiling 减少内存访问, 本讲用 replication/sharding 减少或重排通信。真正的问题不是“用了多少张卡”, 而是每一步训练中数据必须移动多远、移动多少、何时移动。

## 1.1 为什么要多 GPU

多 GPU 的动机可以分成两类。第一类是容量: 模型参数、梯度、optimizer state 和激活放不进一张卡。第二类是速度: 单卡算力太小, 训练 wall-clock time 不可接受, 需要更多 FLOPs。实际大模型训练通常先被容量推着走, 然后再被速度逼着优化并行效率。

$$M_{\text{train}} \approx M_{\text{params}} + M_{\text{grads}} + M_{\text{optimizer}} + M_{\text{activations}}$$

其中  $M_{\text{params}}$  是权重显存,  $M_{\text{grads}}$  是反向传播后的梯度,  $M_{\text{optimizer}}$  是 Adam/AdamW 的一阶动量  $m$  和二阶动量  $v$  等状态,  $M_{\text{activations}}$  是前向保存给反向使用的激活。这个公式不是精确预算, 而

是告诉我们：训练显存不是只由参数量决定。

### 多 GPU 不是自动加速按钮

更多 GPU 只提供潜在算力。若每个 step 的通信量太大、通信无法与计算重叠、或并行策略与硬件拓扑不匹配，实际吞吐可能几乎不涨，甚至因为同步和调度开销变慢。

## 1.2 泛化的层次结构

| 层次       | 典型介质                       | 优化含义                                        |
|----------|----------------------------|---------------------------------------------|
| 单 GPU 片上 | registers、L1、shared memory | fusion、tiling、kernel design，尽量不把中间结果写回 HBM。 |
| 单 GPU 片外 | HBM                        | 提高 arithmetic intensity，减少 HBM traffic。     |
| 单节点多 GPU | NVLink、NVSwitch、PCIe       | 可承载较频繁 collective，但仍比本卡内存慢。                 |
| 多节点多 GPU | InfiniBand、RoCE、Ethernet   | 适合较粗粒度并行，必须减少同步次数并隐藏延迟。                     |

## 1.3 本章小结

Lecture 7 的主线是：把一个训练 step 拆成本地计算、状态放置、通信原语、硬件路径四层来看。后面所有 data parallelism、tensor parallelism、pipeline parallelism 都是这四层的不同组合。

## 2 Collective Operations：分布式训练的通信积木

### 2.1 rank、world size 与 process group

分布式程序通常让每张 GPU 对应一个进程。每个进程有一个 **rank**，表示它在通信组里的编号；**world size** 是通信组里的 rank 总数；**process group** 是一组共同参与通信的 rank。一个模型可能同时有 data-parallel group、tensor-parallel group、pipeline group，因此 rank 的“全局编号”和“某个 group 内编号”要区分清楚。



图 2: 四个 rank 构成一个 world size 为 4 的通信世界。

来源：CS336 Spring 2026 Lecture 7 官方可执行讲义图像。

### 读图：rank 图在说明什么

图中每个方块代表一个参与通信的进程或 GPU。rank 不是“机器号”，也不是“模型层号”，而是 collective API 里用来指定参与者身份的编号。world size 是这个通信世界有多少参与者。读这张图时要注意：同一个物理 GPU 在不同 process group 里可能有不同的局部 rank，因此真实训练框架里常常要同时维护 global rank、local rank、data parallel rank、tensor parallel rank。

## collective 的执行约束

collective operation 是所有参与 rank 共同执行的通信模式。所有 rank 必须以相同顺序进入同一个 collective，并且张量 shape、dtype、op 语义要匹配。某个 rank 少调用一次 `all_reduce`，其他 rank 就可能永久等待。

## 2.2 术语消化：从基础原语到工作马

| Collective     | 做什么                                 | 典型用途                                  | 记忆方式                         |
|----------------|-------------------------------------|---------------------------------------|------------------------------|
| broadcast      | 一个 rank 的完整张量复制给所有 rank。            | rank 0 读取 checkpoint 后同步初始权重。         | 一人发，全员收。                     |
| scatter        | 一个 rank 的张量切成多份，分别发给各 rank。         | 理解 reduce-scatter 的基础。                | 切开再分发。                       |
| gather         | 各 rank 的分片收集到一个 rank。               | 诊断、评估、单点聚合。                           | scatter 的反向。                 |
| reduce         | 各 rank 张量按 sum/min/max 等归约到一个 rank。 | 指标聚合、小规模统计。                           | 先算再收。                        |
| all-gather     | gather 的结果给到所有 rank。                | 参数分片前向前拼回完整参数。                        | gather 到所有人。                 |
| reduce-scatter | 先 reduce，再把结果分片给各 rank。             | ZeRO/FSDP 梯度同步后只保留本 rank 分片。          | all-reduce 的前半段。             |
| all-reduce     | reduce 后所有 rank 都拿到完整结果。            | DDP 梯度同步。                             | reduce-scatter + all-gather。 |
| all-to-all     | 每个 rank 都给每个 rank 发送一部分。            | MoE token routing、expert parallelism。 | 分布式矩阵转置。                     |

## 2.3 broadcast、scatter、gather、reduce

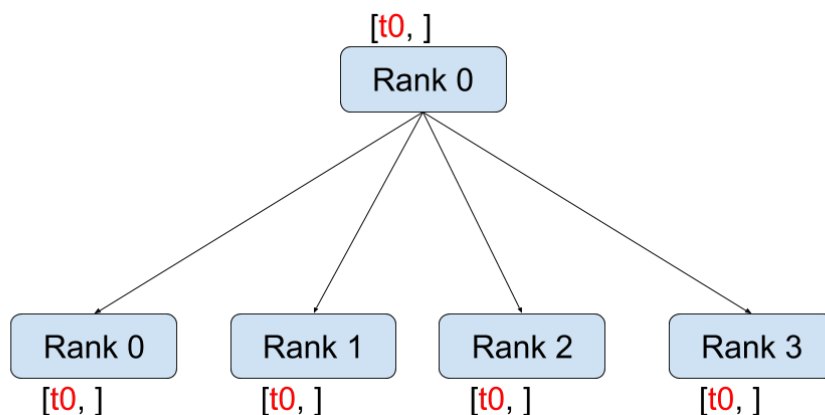


图 3: broadcast: 源 rank 的数据复制到所有 rank。

来源: PyTorch distributed collective diagram, 供 CS336 Lecture 7 配套解释使用。

### 读图：broadcast 的含义

图中只有一个 rank 拥有完整输入，通信后所有 rank 都得到相同副本。这种模式适合“小而关键”的同步，例如初始化权重、广播配置、广播随机种子。它不是训练主路径里最高频的原语，因为频繁广播大张量意味着每一步都在复制状态。

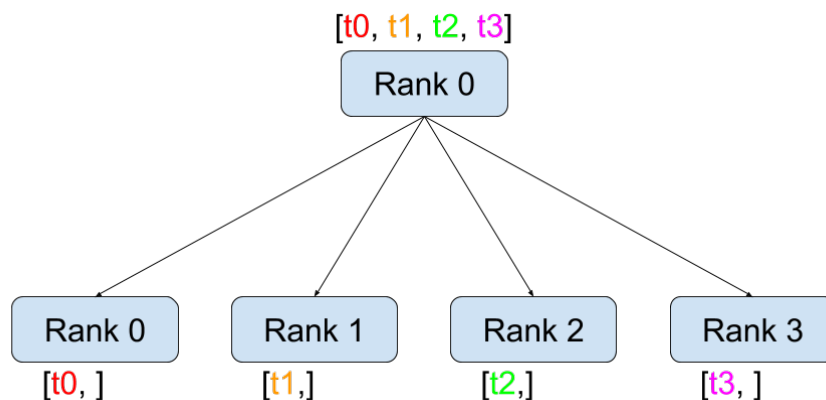


图 4: scatter: 源 rank 把张量切片后分发给不同 rank。

来源: PyTorch distributed collective diagram, 供 CS336 Lecture 7 配套解释使用。

### 读图：scatter 的含义

scatter 的输出不是每个 rank 拿到完整数据，而是每个 rank 拿到自己的切片。它最重要的教学价值是帮我们理解“分片后的所有权”：切片一旦分发出去，后续计算就应该尽量在拥有该切片的 rank 上发生，否则会重新引入通信。

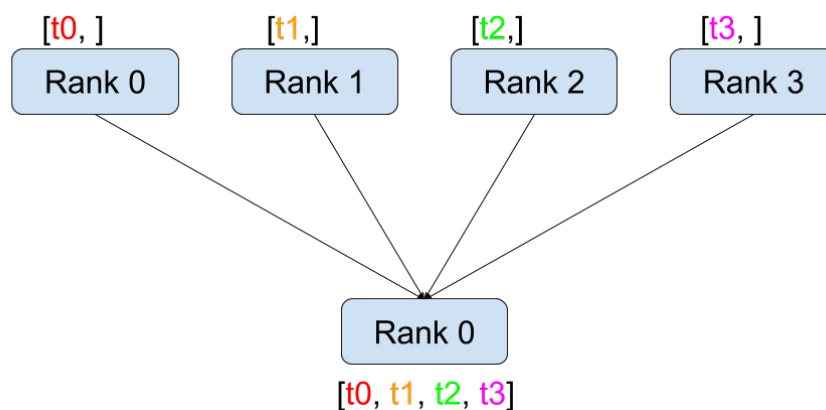


图 5: gather: 各 rank 的分片汇总到一个 rank。

来源: PyTorch distributed collective diagram, 供 CS336 Lecture 7 配套解释使用。

### 读图：gather 的含义

gather 是 scatter 的反向。图中多个 rank 各自持有一片数据，通信后某个目标 rank 拿到完整拼接结果。它在训练主循环中要谨慎使用，因为单个目标 rank 会成为内存和带宽热点；all-gather 则把这个完整结果复制给所有 rank。

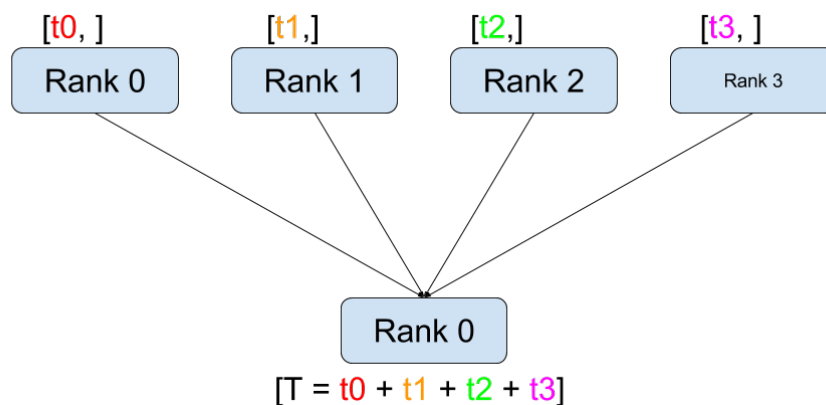


图 6: reduce: 各 rank 的数据按某个操作聚合到一个 rank。

来源: PyTorch distributed collective diagram, 供 CS336 Lecture 7 配套解释使用。

### 读图: reduce 的含义

reduce 不是简单拼接, 而是对对应位置做 sum、min、max 等结合律/交换律操作。训练里最常见的是对梯度求和或平均。若只把结果放到 rank 0, rank 0 后续会成为单点; 因此大规模训练更常用 all-reduce 或 reduce-scatter。

### 基础 collective 的常见误解

broadcast 和 gather 都会产生完整副本, 但语义不同: broadcast 是一份输入复制到多人, gather 是多人输入汇总到一人。scatter 和 reduce-scatter 都让每个 rank 得到分片, 但 reduce-scatter 在分片前还做了跨 rank 归约。

## 2.4 all-gather、reduce-scatter、all-reduce

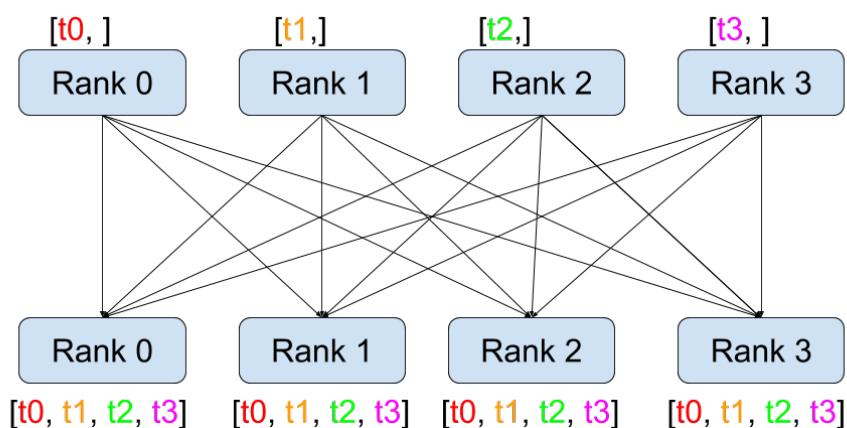


图 7: all-gather: 每个 rank 的分片被拼接, 并且完整结果分发给所有 rank。

来源: PyTorch distributed collective diagram, 供 CS336 Lecture 7 配套解释使用。

如果每个 rank 只保存参数的一部分，那么某一层 forward 可能需要临时看到完整权重。all-gather 的作用就是把各 rank 的参数 shard 拼回来，并让每个 rank 都得到完整参数。它节省的是长期存储，不是通信：每次需要完整参数时，通信仍然会发生。

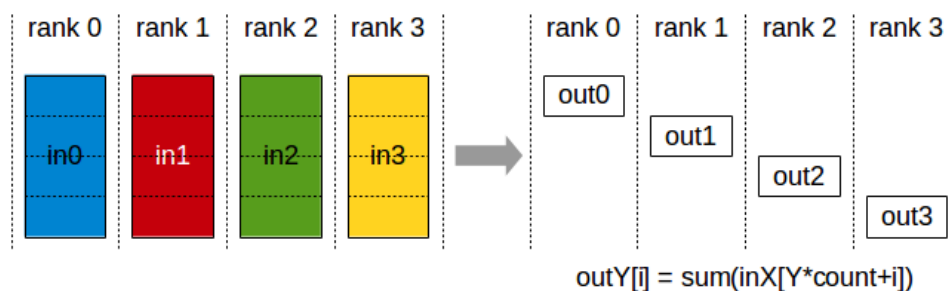


图 8: reduce-scatter: 先跨 rank 归约，再把归约结果切片分发。

来源: PyTorch distributed collective diagram, 供 CS336 Lecture 7 配套解释使用。

### 读图: reduce-scatter 在 ZeRO/FSDP 中做什么

图中每个 rank 起初都有一组向量，通信后每个 rank 只拿到归约结果的一片。把这个逻辑放到训练里：每个 data shard 产生梯度，梯度需要跨 rank 求和；但如果参数或 optimizer state 已经分片，就没有必要让每张卡都保存完整梯度。reduce-scatter 正好把“同步”和“分片保存”合成一步。

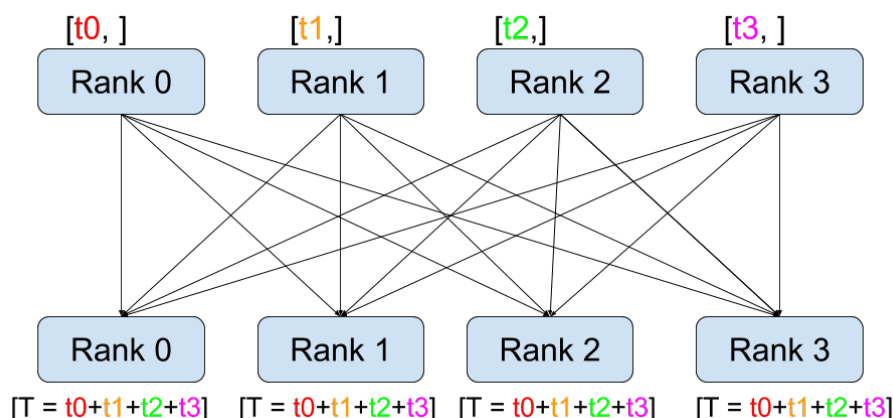


图 9: all-reduce: 归约结果最终复制到所有 rank。

来源: PyTorch distributed collective diagram, 供 CS336 Lecture 7 配套解释使用。

### 读图: all-reduce 是 DDP 的心脏

在 data parallelism 中，每个 rank 使用不同 batch slice 做 forward/backward，因此梯度一开始不同。all-reduce 把所有 rank 的梯度求和或平均后，再把同一份结果交给所有 rank。于是每个 rank 虽然看过不同数据，但参数更新保持一致。

### 核心恒等式

$$\text{all-reduce} = \text{reduce-scatter} + \text{all-gather}.$$

左边的语义是“所有人得到完整归约结果”。右边先让每个人得到归约结果的一片，再把这些片重新拼给所有人。这个拆分是从 DDP 走向 ZeRO/FSDP 的关键：如果后续并不需要每个人长期保存完整结果，就可以停在 reduce-scatter，省掉复制状态。

## 2.5 all-to-all: MoE 的通信核心

all-to-all 是最一般的 collective: 每个 rank 都向每个 rank 发送一段张量。官方讲义强调它对 MoE 很重要, 因为 expert parallelism 中, 每个 rank 可能持有一部分 experts; token 先在本地 batch 中产生, 然后必须被路由到拥有对应 expert 的 rank 上。

### 术语消化: all-to-all 与 MoE

| 术语                 | 机制                                        | 为什么影响训练系统                                   |
|--------------------|-------------------------------------------|---------------------------------------------|
| expert parallelism | 不同 rank 保存不同 experts, token 根据 router 分发。 | 参数容量增加, 但 token routing 触发 all-to-all。      |
| balanced split     | 每个 rank 发出/接收的数据量接近。                      | all-to-all 更像规则矩阵转置, 通信更可预测。                |
| unbalanced split   | 某些 expert 热门, 部分 rank 接收过多 token。         | 慢 rank 决定 step time, load balancing 变成系统问题。 |

### all-to-all 不只是“通信量大”

all-to-all 的难点还包括负载不均、变长 token dispatch、buffer 管理、跨节点拥塞和反向传播中的路由一致性。MoE 的 router 如果只优化模型 loss, 不控制负载, 系统吞吐会被少数热门 experts 拖垮。

## 2.6 本章小结

Collectives 是分布式训练的基础语言。读懂本讲后, 看到 DDP、FSDP、tensor parallel、pipeline parallel、MoE 时, 不应只记名字, 而应追问: 这个策略使用了哪些 collective, 通信对象是什么, 结果是复制还是分片。

## 3 硬件互连: 拓扑决定可承受的通信频率

### 3.1 从家用拓扑到数据中心拓扑

官方讲义先给出一个“classic in the home”的对比: 同一台机器里的 GPU 通过 PCIe 连接, 不同机器通过普通 Ethernet 连接。这个图不是为了讲家庭装机, 而是为了强调: 如果跨机器通信还要经过 CPU 网络栈, 训练系统很快会被数据搬运拖死。

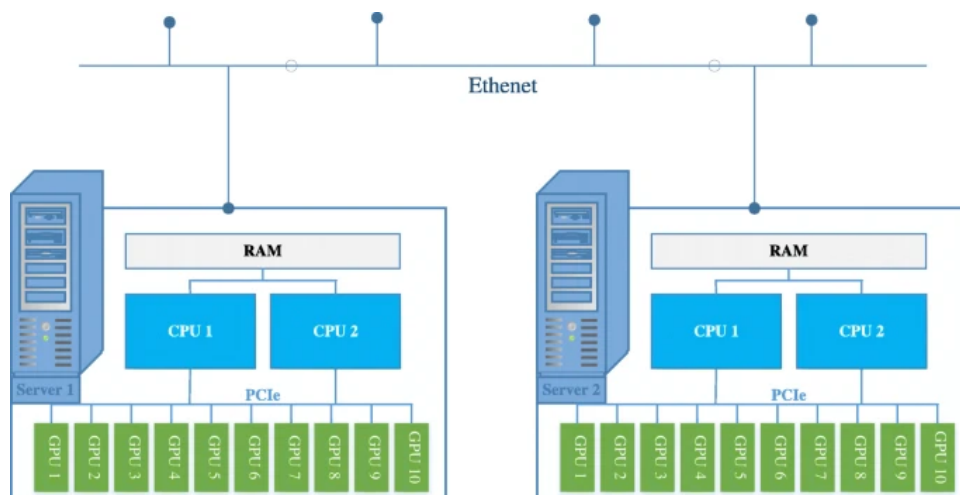


图 10: 传统服务器/网络拓扑示意：设备通过 PCIe 和网络层层连接。

来源：CS336 Lecture 7 官方源引用的 Springerature 图；本地化文件名保留原始标识 42774\_2021\_98\_Fig1\_HTML.png?as=webp。

### 读图：为什么普通拓扑不够训练大模型

这张图的重点是“路径”。GPU 到 GPU 的数据若必须先过 PCIe、CPU 内存、内核网络栈、NIC，再穿过以太网到另一台机器，路径中的每一层都会增加延迟并消耗带宽。大模型训练的同步张量动辄 GB 级，普通网络设计主要服务通用 I/O，不是为每个训练 step 高频搬运梯度和激活而生。

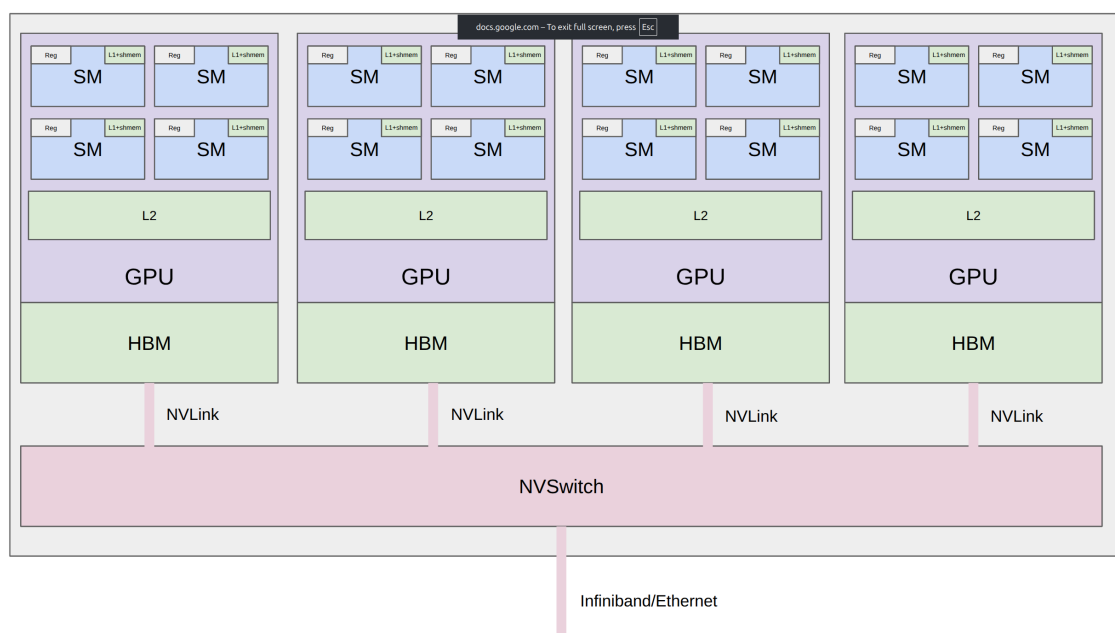


图 11: 数据中心 GPU 节点：NVLink/NVSwitch、HCA/NIC、InfiniBand/RoCE 形成层次化互连。

来源：CS336 Spring 2026 Lecture 7 官方可执行讲义图像。

### 读图：现代拓扑如何支持不同并行策略

图中蓝色 GPU 域适合高频通信，因为 NVLink/NVSwitch 的带宽远高于跨节点链路。节点间通过 HCA/NIC 接入 InfiniBand 或 RoCE，适合 data parallel 的梯度同步、pipeline stage 边界传激活等相对粗粒度通信。读这张图时应把每种并行策略放到拓扑上：tensor parallel 的每层 all-gather/all-reduce 尽量留在节点内，pipeline parallel 的 stage 边界才更可能跨节点。

## 3.2 带宽数量级和路径差异

官方源给出的数量级是：PCIe 7.0 x16 约 242 GB/s，B200 的 NVLink 5.0 节点内可达 TB/s 量级，HBM 约 8 TB/s，而跨节点 InfiniBand 链路可能只有 0.05 TB/s 量级。具体硬件会变化，但排序稳定：片上/本卡最快，节点内互连其次，跨节点最慢。

$$T_{\text{comm}} \approx \alpha + \frac{S}{B_{\text{effective}}},$$

其中  $T_{\text{comm}}$  是一次通信时间， $\alpha$  是启动延迟和同步开销， $S$  是需要移动的数据量， $B_{\text{effective}}$  是实际有效带宽。小张量通信常被  $\alpha$  主导，大张量通信常被带宽主导。

### 不要把规格表带宽当成训练吞吐

规格表数字通常是单链路或理想条件下的峰值。训练中还有拓扑竞争、协议开销、NCCL algorithm 选择、张量大小、同步点、反向传播依赖等因素。并行策略选择必须用实际 benchmark 校准。

## 3.3 RDMA、InfiniBand 与 RoCE

RDMA 是 Remote Direct Memory Access，意思是一台机器可以在较少 CPU 介入的情况下直接读写另一台机器的内存或 GPU 相关 buffer。InfiniBand 原生支持 RDMA；RoCE 是 RDMA over Converged Ethernet，在以太网上提供类似能力，但部署和拥塞控制复杂度不同。对训练来说，绕过 CPU 的价值在于减少数据拷贝和内核网络栈开销，让 GPU 间同步更接近“设备到设备”的路径。

### 硬件抽象的边界

NCCL 和 PyTorch 可以隐藏 API 细节，但不能消除物理层级。一个每层都通信的 tensor-parallel 模型放到慢跨节点链路上，仍然会慢；一个 pipeline stage 切得不均匀，即使网络很快，也会被最慢 stage 决定吞吐。

## 3.4 NVIDIA Collective Communication Library (NCCL)

NCCL, 即 NVIDIA Collective Communication Library (NCCL), 把 all-reduce、all-gather、reduce-scatter 等 collective 映射到底层 GPU 通信路径。它会检测拓扑，选择 ring/tree 等算法，启动 GPU kernel 来收发数据。PyTorch 的 `torch.distributed` 在 GPU 场景下常用 NCCL backend。

## 术语消化: NCCL 负责什么, 不负责什么

| 层次   | NCCL 能做                          | NCCL 不能替你做           |
|------|----------------------------------|----------------------|
| 拓扑感知 | 识别 GPU、NVLink、PCIe、NIC 的连接关系。    | 改变硬件带宽或跨节点物理距离。      |
| 算法选择 | 为 collective 选择 ring、tree 等通信算法。 | 减少模型本身要求的同步语义。       |
| 执行路径 | 用 GPU kernel 发送/接收数据, 减少 CPU 干预。 | 自动决定模型应该怎么 sharding。 |

### 3.5 本章小结

并行策略不是抽象数学题, 而是拓扑选择题。通信频率越高, 越要靠近高速互连; 通信越粗粒度, 越能跨慢链路。硬件决定“哪些策略值得尝试”, benchmark 决定“具体怎么切”。

## 4 torch.distributed: 把 collective 写成代码

### 4.1 初始化和 process group

官方讲义用 `torch.multiprocessing.spawn` 启动多个进程, 每个进程运行同一个函数, 但传入不同 rank。然后用 `dist.init_process_group` 建立通信环境。若有 GPU, 就用 NCCL; 否则用 Gloo。

Listing 1: Lecture 7 中分布式初始化的最小结构

```
1 def setup(rank: int, world_size: int):
2     os.environ["MASTER_ADDR"] = "localhost"
3     os.environ["MASTER_PORT"] = "15623"
4
5     if torch.cuda.is_available():
6         dist.init_process_group("nccl", rank=rank, world_size=world_size)
7     else:
8         dist.init_process_group("gloo", rank=rank, world_size=world_size)
```

### 一个 rank 通常对应一个进程

在 PyTorch DDP 风格中, 常见做法是 one process per GPU。这样每个进程只管理一张卡, CUDA context、随机种子、通信 rank 都更清晰。多机训练时还要区分 local rank 和 global rank: local rank 决定本机第几张 GPU, global rank 决定全局通信身份。

### 4.2 all-reduce 示例

官方代码中, 每个 rank 先构造一个不同的向量:

Listing 2: all-reduce 会原地修改每个 rank 的 tensor

```
1 data = tensor([0., 1, 2, 3], device=cuda_if_available(rank)) + rank
2 dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
```

若 world size 为 4，四个输入分别是 [0, 1, 2, 3]、[1, 2, 3, 4]、[2, 3, 4, 5]、[3, 4, 5, 6]，sum all-reduce 后每个 rank 都得到 [6, 10, 14, 18]。这正是 DDP 梯度同步的形状：每张卡最初有自己的梯度，通信后每张卡有相同平均梯度。

### 4.3 reduce-scatter + all-gather 示例

Listing 3: reduce-scatter 后接 all-gather 复原 all-reduce 语义

```
1 input = torch.arange(world_size, dtype=torch.float32,
2                       device=cuda_if_available(rank)) + rank
3 output = torch.empty(1, device=cuda_if_available(rank))
4 dist.reduce_scatter_tensor(output=output, input=input,
5                             op=dist.ReduceOp.SUM, async_op=False)
6
7 input = output
8 output = torch.empty(world_size, device=cuda_if_available(rank))
9 dist.all_gather_into_tensor(output_tensor=output,
10                             input_tensor=input, async_op=False)
```

#### 这段代码为什么是后续课程的桥

如果做完 reduce-scatter 后就停止，每个 rank 只保留归约结果的一片；如果再做 all-gather，每个 rank 又得到完整归约结果。ZeRO/FSDP 的很多设计就在这两个点之间选择：什么时候需要完整参数，什么时候只保存分片状态。

### 4.4 barrier、同步和 deadlock

`dist.barrier()` 让所有 rank 等到同一个同步点。它常用于 benchmark、调试打印、确保 warmup 完成。它也会暴露控制流不一致：如果某个 rank 进入了 barrier，另一个 rank 因条件分支跳过，程序就会卡住。

#### 分布式 deadlock 常来自不一致

常见错误包括：某个 rank 少调用一次 collective、不同 rank 的 tensor shape 不一致、某些 rank 提前异常退出、print 或 dataloader 导致顺序错位。调试时先确认所有 rank 的 collective 顺序完全一致，再看性能。

### 4.5 本章小结

`torch.distributed` 的 API 看起来只是函数调用，但每个调用背后都是多进程同步协议。写分布式训练代码时，正确性约束包括数学结果、shape/dtype、调用顺序和进程生命周期。

## 5 通信 Benchmark：测的是带宽，也是算法

### 5.1 为什么要单独测 collective

官方讲义的 benchmark 不是为了得到一个漂亮数字，而是训练读者形成判断：某个集群上 all-reduce、reduce-scatter 的实际速度是多少，是否足以支持当前并行策略。若 benchmark 已经显示通信占比过高，继续调学习率或 batch size 都解决不了根因。

Listing 4: all-reduce benchmark 的关键步骤

```
1 data = torch.randn(num_elements, device=cuda_if_available(rank))
2 dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
3 torch.cuda.synchronize()
4 dist.barrier()
5
6 start_time = time.time()
7 dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
8 torch.cuda.synchronize()
9 dist.barrier()
10 duration = time.time() - start_time
```

### 5.2 带宽估算公式

官方代码用一个环形通信直觉估算 all-reduce 的有效带宽：

$$B_{\text{all-reduce}} = \frac{2S(N-1)}{NT},$$

其中  $S$  是单 rank 张量大小， $N$  是 world size， $T$  是一次 all-reduce 的 wall-clock duration。分子里的 2 来自 ring all-reduce 中 reduce-scatter 与 all-gather 两段都要传输数据； $(N-1)$  表示每个 rank 需要参与多个传输步；分母的  $NT$  是把全局传输工作量归一到整体通信时间上的一种有效带宽口径。

reduce-scatter 的估算少一个 2：

$$B_{\text{reduce-scatter}} = \frac{S_{\text{input}}(N-1)}{NT}.$$

这里  $S_{\text{input}}$  是每个 rank 输入矩阵的总字节数。两者带宽可能相近，因为 all-reduce 可以看成 reduce-scatter 加 all-gather：通信量和时间都约翻倍，带宽口径未必翻倍。

#### 不要只看 duration

同一个 collective 的 duration 会随 tensor size、world size、拓扑、NCCL 算法、warmup、同步方式变化。报告性能时至少要同时给出 bytes、world size、dtype、硬件拓扑和有效带宽。

### 5.3 常见测量陷阱

#### benchmark 的四个坑

1. 没有 warmup：第一次通信可能包含初始化、连接建立、kernel 编译或缓存效应。
2. 没有 `torch.cuda.synchronize()`：CUDA 异步执行会让计时只测到 launch time。
3. 没有 barrier：不同 rank 开始/结束时间不一致，测出来的不是集体通信。

4. 只测一个 tensor size: 小消息看 latency, 大消息看 bandwidth, 两个区间结论不同。

## 5.4 本章小结

通信 benchmark 是并行策略选择的输入。它回答的问题不是“这台机器快不快”，而是“这个 collective 在这个张量大小和拓扑上是否足够快，能否放进训练 step 的关键路径”。

# 6 Data Parallelism: 沿 batch 维切分

## 6.1 基本策略

Data parallelism 沿 batch dimension 切分数据。每个 rank 保留完整模型参数，读取自己的 batch slice，独立 forward/backward，然后用 all-reduce 同步梯度。同步后每个 rank 执行相同 optimizer step，因此参数保持一致。

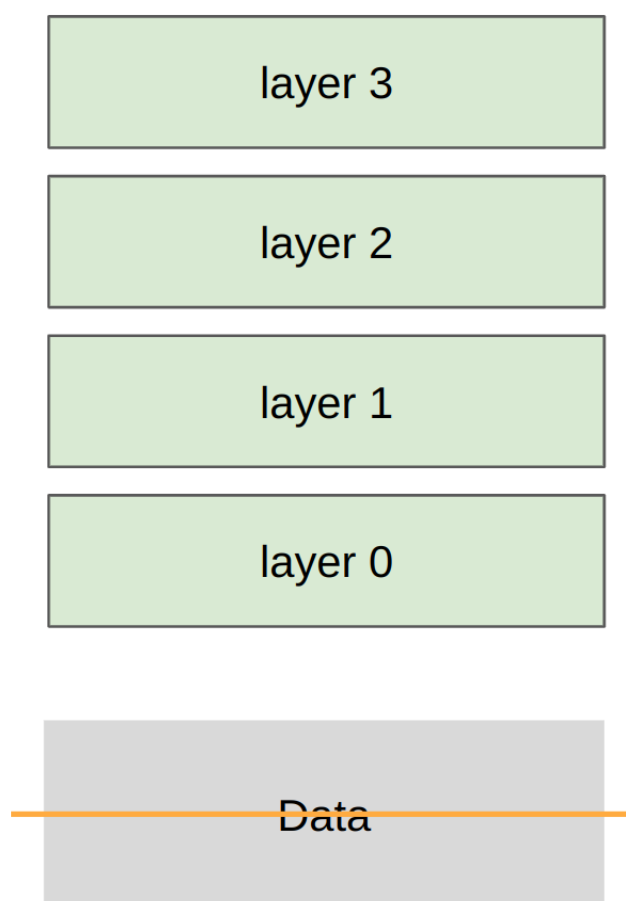


图 12: Data parallelism: 切分 batch, 复制模型。

来源: CS336 Spring 2026 Lecture 7 官方可执行讲义图像。

### 读图：data parallel 图里的“复制”和“切分”

图中每个 rank 都有完整网络层，所以参数、梯度和 optimizer state 默认是复制的；被切分的是 batch。这个策略的好处是每张卡可以独立做完整 forward/backward，通信只集中在梯度同步阶段。坏处是显存不随 GPU 数量线性扩展，因为模型状态仍然每卡一份。

Listing 5: Lecture 7 的 data parallel 核心：本地 loss，跨 rank 平均梯度

```
1 local_batch_size = batch_size // world_size
2 start_index = rank * local_batch_size
3 data = data[start_index:start_index + local_batch_size].to(device)
4
5 for param in params:
6     x = x @ param
7     x = F.gelu(x)
8 loss = x.square().mean()
9 loss.backward()
10
11 for param in params:
12     dist.all_reduce(tensor=param.grad, op=dist.ReduceOp.AVG,
13                     async_op=False)
```

## 6.2 通信账本

若模型参数总量为  $P$  bytes，每一步 DDP 梯度同步的主通信对象也是  $P$  bytes 级别的梯度。由于每个 rank 都要得到完整平均梯度，典型实现使用 all-reduce。计算量随本地 batch 增长，通信量主要随参数量增长，因此 data parallel 对“大 batch、较厚计算”的场景友好。

### Data parallel 的优点和代价

优点是实现简单、数学语义接近单机大 batch、通信集中。代价是每卡保存完整参数、梯度和 optimizer state；若单卡装不下模型，普通 DDP 无法解决容量问题，只能引入 ZeRO/FSDP 这类状态分片方法。

## 6.3 为什么引出 FSDP/ZeRO

官方源写道：下一次会讲 FSDP/ZeRO，用 all-gather 和 reduce-scatter 避免每张卡都持有全部参数。直觉是：

- forward 某层前，用 all-gather 临时拼出该层完整参数；
- backward 后，用 reduce-scatter 同步梯度并只保留本 rank 负责的分片；
- optimizer state 也按 rank 分片，减少重复存储。

## 术语消化：DDP、FSDP、ZeRO 的差别

| 方法         | 保存方式                                    | 通信方式与课程关系                                    |
|------------|-----------------------------------------|----------------------------------------------|
| DDP        | 每卡完整参数、梯度、optimizer state。              | backward 后 all-reduce 梯度，简单但显存重复。            |
| ZeRO-1/2/3 | 按 stage 分片 optimizer state、梯度、参数。       | 用 reduce-scatter/all-gather 把复制状态改为分片状态。     |
| FSDP       | PyTorch 中的 fully sharded data parallel。 | 以模块为单位 all-gather 参数、释放参数、reduce-scatter 梯度。 |

### 6.4 本章小结

Data parallelism 是最容易理解的并行策略：数据切开，模型复制，梯度同步。它扩展吞吐，但不天然扩展模型容量；状态分片才是解决大模型显存的关键。

## 7 Tensor Parallelism：沿 width 维切分

### 7.1 基本策略

Tensor parallelism 沿层内部张量维度切分，例如把矩阵乘法的输出宽度切给多个 rank。每个 rank 保存该层参数的一部分，计算一部分激活，然后通过 all-gather 或 all-reduce 恢复下一层需要的完整张量。

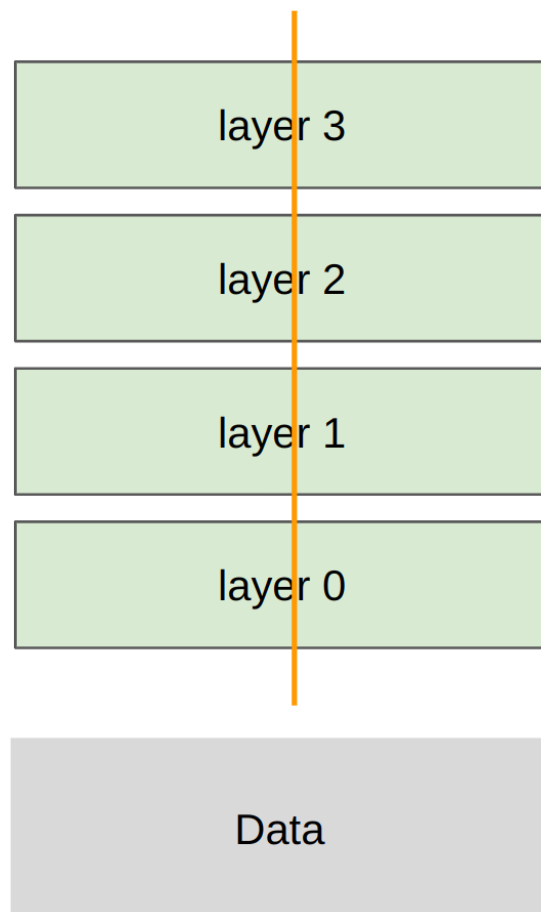


图 13: Tensor parallelism: 切分 layer width, 每层内跨 rank 协作。

来源: CS336 Spring 2026 Lecture 7 官方可执行讲义图像。

#### 读图: tensor parallel 图说明的不是“多放几层”

图中每一层都被横向拆开, 不同 rank 负责同一层的不同列或通道。这样可以让单层参数和计算分摊到多张卡上, 但下一层往往需要完整激活, 因此每层都可能发生通信。读图时要把它和 pipeline parallel 区分: tensor parallel 是同一层多卡一起算, pipeline parallel 是不同层放在不同卡上。

Listing 6: Lecture 7 的 tensor parallel 前向: 局部 matmul 后 all-gather 激活

```

1 local_num_dim = num_dim // world_size
2 params = [get_init_params(num_dim, local_num_dim, rank)
3           for layer in range(num_layers)]
4
5 x = data
6 for layer in range(num_layers):
7     x = x @ params[layer]
8     x = F.gelu(x)
9     activations = [
10         torch.empty(batch_size, local_num_dim, device=device)
11         for _ in range(world_size)
12     ]
13     dist.all_gather(tensor_list=activations, tensor=x,
```

```
14         async_op=False)
15     x = torch.cat(activations, dim=1)
```

## 7.2 列切、行切和通信位置

设输入  $X \in \mathbb{R}^{B \times d}$ ，权重  $W \in \mathbb{R}^{d \times h}$ 。若按输出宽度切分  $W = [W_1, W_2, \dots, W_N]$ ，rank  $i$  计算：

$$Y_i = XW_i, \quad Y = [Y_1, Y_2, \dots, Y_N].$$

其中  $B$  是 batch tokens 数， $d$  是输入 hidden size， $h$  是输出 hidden size， $N$  是 tensor-parallel world size。每个 rank 只算  $Y_i$ ，但若下一层需要完整  $Y$ ，就要 all-gather。Megatron-LM 风格会交替使用 column-parallel 和 row-parallel linear，把某些通信推迟或合并，但基本账本不变：切 layer width 就要在层边界恢复一致语义。

### Tensor parallel 的通信频率很高

Tensor parallel 的通信常发生在每个 Transformer block 的多个线性层或 attention 子层边界。它适合放在 NVLink/NVSwitch 这类高速节点内互连上；跨节点使用时，通信延迟和带宽很容易压过局部 matmul 的收益。

## 7.3 反向传播为什么更复杂

官方源把 backward 留作 homework，是有意的：前向只要拼激活，反向还要处理梯度如何在切分维度上归约。列切时，输出梯度对应各自分片；行切时，输入梯度通常需要 reduce。真实框架会把这些规则封装到 parallel linear layers 里，但工程师仍要理解通信发生在哪个方向。

### Tensor parallel 的正确性边界

不能随意把任意 tensor 切开就叫 tensor parallel。切分维度必须与算子数学结构兼容；否则虽然张量 shape 能拼回去，结果也可能不等价，或反向梯度不正确。

## 7.4 本章小结

Tensor parallelism 解决单层太宽、单卡装不下或算不快的问题。它牺牲的是高频通信，因此硬件要求最高。判断是否适合 tensor parallel，要先问每层通信能否被节点内高速互连承受。

# 8 Pipeline Parallelism: 沿 depth 维切分

## 8.1 基本策略

Pipeline parallelism 沿层深度切分模型：前几层放在 rank 0，中间层放在 rank 1，后几层放在后续 rank。每个 rank 只保存自己的 layers，forward 时把激活从前一 stage 发送到后一 stage。

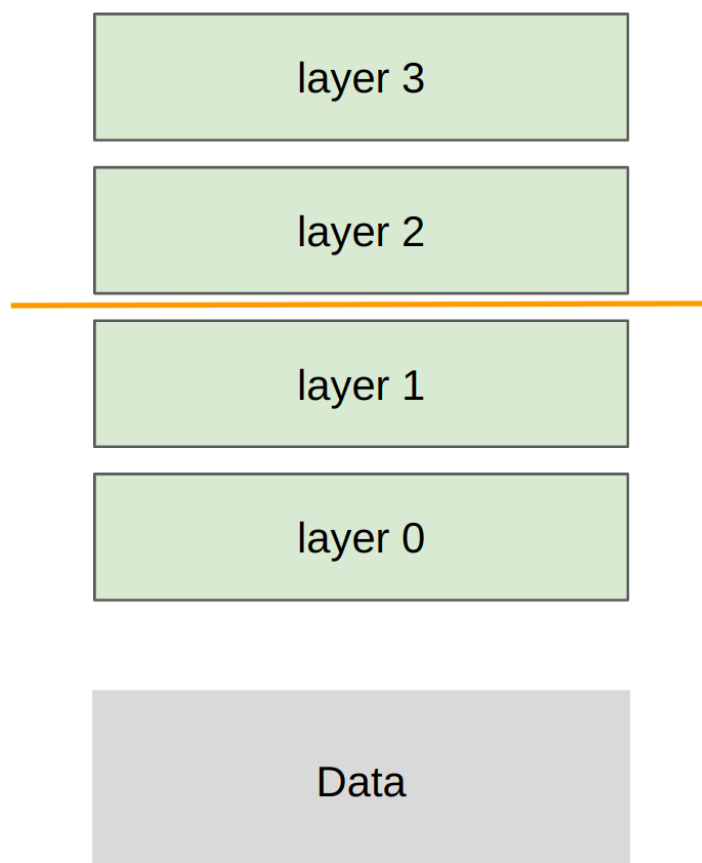


图 14: Pipeline parallelism: 切分 layer depth, 不同 rank 保存不同层段。

来源: CS336 Spring 2026 Lecture 7 官方可执行讲义图像。

#### 读图: pipeline parallel 图里的 stage 和 bubble

图中每张卡负责连续层段, 数据像流水线一样从前向后流动。优点是通信只发生在 stage 边界, 频率低于 tensor parallel; 缺点是流水线刚开始和结束时部分 stage 空闲, 这就是 bubble。microbatch 的作用是把一个大 batch 切小, 让不同 microbatch 同时处在不同 stage, 从而填满流水线。

Listing 7: Lecture 7 的 pipeline 前向: microbatch + send/recv

```

1 micro_batch_size = batch_size // num_micro_batches
2 if rank == 0:
3     micro_batches = data.chunk(chunks=num_micro_batches, dim=0)
4 else:
5     micro_batches = [
6         torch.empty(micro_batch_size, num_dim, device=device)
7         for _ in range(num_micro_batches)
8     ]
9
10 for x in micro_batches:
11     if rank - 1 >= 0:
12         dist.recv(tensor=x, src=rank - 1)
13

```

```

14     for param in local_params:
15         x = F.gelu(x @ param)
16
17     if rank + 1 < world_size:
18         dist.send(tensor=x, dst=rank + 1)

```

## 8.2 bubble 的直觉公式

若有  $P$  个 pipeline stages 和  $M$  个 microbatches, 最简单 GPipe 式调度的前向 bubble 比例近似为:

$$\rho_{\text{bubble}} \approx \frac{P-1}{M+P-1}.$$

其中  $P-1$  是填满流水线需要等待的 stage 数,  $M$  是实际输入的 microbatch 数。增加  $M$  可以降低 bubble, 但 microbatch 太小会降低 kernel efficiency, 也会增加调度和通信次数。

### Pipeline 的核心瓶颈是调度, 而不只是带宽

Pipeline parallel 的通信对象通常是 stage 边界激活, 频率低于 tensor parallel。但它引入调度问题: stage 时间不均、forward/backward 依赖、microbatch 数、activation 保存或重算策略, 都会影响吞吐。

## 8.3 为什么 pipeline 能跨较慢链路

Pipeline stage 之间的通信发生在层段边界, 粒度较粗。如果每个 stage 内部有足够多层和足够大计算, 通信可以被较长的 compute window 吸收。因此 pipeline parallel 更可能跨节点使用。但这要求模型切分均衡: 每个 stage 的参数数量、计算量、激活大小和 backward 时间都要接近。

### Pipeline parallel 不自动负载均衡

如果某个 stage 包含更重的 attention 或更宽的 MLP, 它会成为全局吞吐瓶颈。真实系统常需要按 profiling 结果手工或自动调整层分配, 而不是简单平均层数。

## 8.4 本章小结

Pipeline parallelism 解决深层模型容量和跨节点放置问题。它通信频率较低, 但要付出 bubble、调度复杂度和 stage imbalance 的代价。

# 9 三种并行策略的比较与组合

## 9.1 切分对象决定通信形态

| 并行方式             | 切分对象  | 优点                              | 主要代价                          |
|------------------|-------|---------------------------------|-------------------------------|
| Data parallelism | batch | 简单、吞吐扩展好、每卡完整 forward/backward。 | 模型状态复制, 梯度 all-reduce 随参数量增长。 |

| 并行方式                        | 切分对象                           | 优点                | 主要代价                                |
|-----------------------------|--------------------------------|-------------------|-------------------------------------|
| Tensor parallelism          | layer width / tensor dimension | 单层参数和计算可跨卡分摊。     | 每层高频通信，强依赖 NVLink/NVSwitch。         |
| Pipeline parallelism        | layer depth                    | 模型深度可跨卡放置，通信较粗粒度。 | pipeline bubble、调度复杂、stage 负载均衡困难。  |
| Sequence/expert parallelism | sequence 或 experts             | 支持长上下文或 MoE 参数扩展。 | attention/expert routing 通信和负载均衡复杂。 |

### 选择并行策略的第一原则

先看瓶颈是什么：如果单卡放不下状态，考虑 ZeRO/FSDP 或 pipeline；如果单层太宽，考虑 tensor parallel；如果吞吐不够但单卡放得下，先做 data parallel；如果 MoE expert 太多，考虑 expert parallel 和 all-to-all 负载均衡。

## 9.2 recompute、memory、communication 三角

官方总结里有一句非常好的工程判断：可以 recompute，可以存在本地 memory，也可以存在其他 GPU 的 memory 然后 communicate。三者是同一个问题的三个出口。

### 术语消化：三角权衡

| 选择            | 省了什么                | 付出什么                    |
|---------------|---------------------|-------------------------|
| memory        | 不重算，不通信，最快读取。       | 占 HBM，单卡容量压力大。          |
| recompute     | 少存激活或中间结果。          | 增加 FLOPs，可能拉长 backward。 |
| communication | 把状态放到别的 rank 或分片存储。 | 消耗网络带宽，增加同步依赖。          |

## 9.3 JAX/TPU 路线为什么看起来不同

官方源提到 JAX/TPU 路线：定义模型、定义 sharding strategy，编译器负责很多底层映射。PyTorch 课程选择从 primitives 讲起，是为了让读者看见底层机制。两者不是谁高谁低，而是抽象层不同：高层 sharding API 更省心，但当性能异常时，仍然需要理解 collectives、拓扑和通信账本。

### 抽象越高，越不能忘记通信账本

自动 sharding 可以搜索或生成并行计划，但目标函数仍然受物理通信约束。性能调试时，如果不知道某个 tensor 被 all-gather 了几次、某个 expert routing 是否 all-to-all，就很难解释 step time。

## 9.4 本章小结

Data、tensor、pipeline 不是互斥选项。真实大模型训练常把它们组合成 3D/4D parallelism：节点内 tensor parallel，节点间 data/FSDP，模型深度再 pipeline，MoE 层另加 expert parallel。组合

的核心仍然是让高频通信留在快链路，让低频通信跨慢链路。

## 10 工程 Checklist：写分布式训练代码前先问什么

### 10.1 状态在哪里

第一组问题是状态放置：

- 参数、梯度、optimizer state、activation 是否复制？
- 哪些对象被 sharding？按 batch、width、depth、sequence 还是 expert？
- 哪个时刻需要完整副本？这个副本是长期保存还是临时 all-gather？

### 10.2 通信何时发生

第二组问题是通信时序：

- collective 是 all-reduce、reduce-scatter、all-gather 还是 all-to-all？
- 通信发生在每层、每个 block、每个 microbatch，还是每个 step？
- 是否能与 backward compute overlap？
- 是否有 barrier 或隐式同步破坏 overlap？

### 10.3 硬件是否匹配策略

第三组问题是拓扑匹配：

- 高频通信是否留在 NVLink/NVSwitch 域内？
- 跨节点链路是否只承载较粗粒度通信？
- NCCL benchmark 的实际带宽是否支持理论计划？
- 最慢 rank 或最慢 stage 在哪里？

#### 最终判断标准

一个并行策略的好坏不由名字决定，而由每个 step 的端到端吞吐、显存峰值、扩展效率和稳定性决定。可解释的系统账本比“用了某某 parallelism”更重要。

### 10.4 本章小结

写分布式训练代码前，先画状态图和通信图。状态图告诉你什么被复制、什么被分片；通信图告诉你每个同步点移动什么张量、走什么链路。没有这两张图，调参只是猜。

## 11 拓展阅读

- [Stanford CS336 Spring 2026 course page](#)
- [PyTorch distributed documentation](#)
- [NVIDIA NCCL user guide](#)
- [NCCL tests performance notes](#)
- [Sample all-reduce benchmark code](#)
- [Levanter: JAX-based distributed training stack from CRFM](#)

## 12 Summary: 总结与延伸

Lecture 7 的 Summary 可以压缩成一句话：分布式训练不是“把模型丢到很多 GPU 上”，而是用 collective operations 在硬件拓扑上实现某种 sharding/replication 计划。Data parallel 沿 batch 切，tensor parallel 沿 width 切，pipeline parallel 沿 depth 切；它们分别改变通信对象、通信频率和状态放置。

### 最终 takeaways

1. 多 GPU 的两个动机是容量和速度：放不下，或想更快。
2. Collective operations 是分布式训练的基础语言；all-reduce、reduce-scatter、all-gather 是工作马。
3. all-reduce = reduce-scatter + all-gather 是理解 DDP 到 FSDP/ZeRO 的关键。
4. Tensor parallel 通信频繁，通常要求 NVLink/NVSwitch；pipeline parallel 通信较粗粒度，但要处理 bubble。
5. 真实训练系统是在 recompute、memory 和 communication 之间换瓶颈。
6. 分布式优化必须用 benchmark/profile 验证，不能只按硬件规格表推断。

### 和下一讲的连接

本讲先搭起 collectives 和三种基本 parallelism。下一步自然是更系统地处理模型状态分片：ZeRO/FSDP 如何用 all-gather 和 reduce-scatter 降低 replicated state，以及怎样通过 overlap/prefetch 把通信藏到计算后面。